

Let's Expand our project

Now our blog application allow user to comment on blogs.

1. Let's first create a comment Schema. it should store the comment message, userid for user information who have added the comment and a blogId for blog information on which blog user have commented. Schema will look something like this

```
model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String?
  createdAt DateTime @default(now())
  blogs   Blog[]
  comments Comment[] // user can have many comments
}

model Blog {
  id      Int      @id @default(autoincrement())
  published Boolean @default(false)
  title   String   @db.VarChar(255)
  body    String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  userId  Int
  user    User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  comments Comment[]
}

model Comment {
  id      Int      @id @default(autoincrement())
  message String
  createdAt DateTime @default(now())
  blogId  Int
  blog    Blog     @relation(fields: [blogId], references: [id], onDelete: Cascade)
  userId  Int
  user    User     @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
}
```

User and comment have one to many relation and blog and comment also has one to many relation.

```
//postgresql query
CREATE TABLE "Comment" (
  id SERIAL PRIMARY KEY,
  message TEXT NOT NULL,
  "createdAt" TIMESTAMP NOT NULL DEFAULT NOW(),
  "blogId" INTEGER NOT NULL,
  "userId" INTEGER NOT NULL,
  CONSTRAINT fk_comment_blog FOREIGN KEY ("blogId") REFERENCES "Blog"(id) ON DELETE CASCADE,
  CONSTRAINT fk_comment_user FOREIGN KEY ("userId") REFERENCES "User"(id) ON DELETE CASCADE
);
CREATE INDEX idx_comment_blogId ON "Comment" ("blogId");
CREATE INDEX idx_comment_userId ON "Comment" ("userId");
```

Now we all know , what should be the next step -

2. Run migration to sync with database and generate clients.

```
npx prisma migrate dev --name "create comment model and mapped with user and blog"
npx prisma generate
```

3. Write a function that **fetches all blogs written by a specific user given userId**, along with:

- The user's details (name, email)
- All comments on each blog
- Each comment's user details (name, email)

```

async function getUserBlogsWithComments(userId) {
  const blogs = await prisma.blog.findMany({
    where: {
      userId: userId
    },
    select: {
      title: true,
      body: true,
      user: {           // blog user details
        select: {
          name: true,
          email: true
        }
      },
    },
    comments: {         // comments on each blog
      select: {
        message: true,
        createdAt: true,
        user: {         // comment user details
          select: {
            name: true,
            email: true
          }
        }
      }
    }
  })

  return blogs;
}

```

```

getUserBlogsWithComments(1)
  .then((data)=>console.log(data))
  .catch((e)=>console.log(e));

```

```

//postgresql query for the same
SELECT

```



```
b.id AS blog_id,  
b.title,  
b.body,  
u.name AS blog_user_name,  
u.email AS blog_user_email,  
c.id AS comment_id,  
c.message AS comment_message,  
c."createdAt" AS comment_created_at,  
cu.name AS comment_user_name,  
cu.email AS comment_user_email  
FROM "Blog" b  
JOIN "User" u ON b."userId" = u.id  
LEFT JOIN "Comment" c ON c."blogId" = b.id  
LEFT JOIN "User" cu ON cu.id = c."userId"  
WHERE b."userId" = 1  
ORDER BY b.id, c."createdAt";
```

Composite keys